

Capability-Preference Decomposition for Adaptive Multi-Tenant LLM Routing

Sriram M

Dept. of AI and Data Science
Kongu Engineering College
Perundurai, India
sriramm.23aid@kongu.edu

Senthilvadivu K

Dept. of AI and Data Science
Kongu Engineering College
Perundurai, India
senthilvadivuk.ai@kongu.edu

Sathya D

Dept. of AI and Data Science
Kongu Engineering College
Perundurai, India
sathyad.ai@kongu.edu

Ramya G

Dept. of AI and Data Science
Kongu Engineering College
Perundurai, India
ramyag.23aid@kongu.edu

Sivaprakash M P

Dept. of AI and Data Science
Kongu Engineering College
Perundurai, India
sivaprakashmp.23aid@kongu.edu

Abstract—Multi-tenant API gateways must balance quality, latency, and cost when routing queries across large language model (LLM) providers. Because these objectives vary across tenants and change over time, routing policies must adapt online. Standard contextual-bandit approaches handle adaptation by collapsing multidimensional provider outcomes into a scalar utility before updating their parameters. This design couples the provider’s estimated capability to whichever tenant preferences were active during training, forcing the system to relearn capabilities from scratch whenever a tenant’s priorities shift. We present Transfer-Learned LinUCB (TL-LinUCB), an algorithm that separates these quantities. TL-LinUCB estimates provider capability as a vector directly from raw outcome observations. Tenant preferences enter the decision purely as a linear projection at routing time. To address tenant cold-start, the algorithm blends a global ridge-regression estimator with a tenant-specific one, weighting them by their relative directional uncertainty. Evaluated across 30 random seeds against five synthetic provider profiles, this decomposition reduces cumulative pseudo-regret by 13.4% compared to a scalar monolithic baseline. When tenant preferences drift abruptly, TL-LinUCB cuts regret by 19.8% against a reset scalar policy because the preference shift requires only a new projection, not capability re-estimation. Finally, we empirically identify a boundary condition where exploration overhead erases the adaptive benefit, demonstrating when static routing remains the superior choice.

Index Terms—LLM routing, contextual bandits, multi-tenant systems, multi-objective optimization, online learning

I. INTRODUCTION

LLM provider routing requires navigating shifting trade-offs. Quality, latency, and cost vary not only by provider but also by query complexity and real-time network conditions. Frontier models execute complex reasoning reliably but incur latency and token costs that lighter models avoid. Consequently, an API gateway processing a heterogeneous query stream cannot statically assign traffic to a single endpoint. The optimal choice depends on the specific query, current provider availability, and the workload mix. Since these conditions fluctuate, the routing logic must adapt online [9], [15].

Multi-tenant deployments introduce an additional requirement: varying objectives. A compliance tenant might prioritize output accuracy above all else, while a high-throughput pipeline optimizes heavily for cost. These tenants share the same gateway, and their individual service-level agreements can evolve. Effective routing therefore requires tracking two distinct quantities: what a provider will likely deliver for a given query (capability), and how much the tenant values each outcome dimension (preference). Capability is a property of the provider and the input; preference is a property of the tenant.

Contextual bandits [1]–[3] offer a natural framework for online adaptation, but they typically scalarize the multi-dimensional outcome before taking a parameter update step. When an estimator pools observations from tenants with conflicting objectives, the gradient steps interfere. The resulting model serves no tenant optimally. Furthermore, any shift in a tenant’s preference vector forces the system to discard the existing parameter state and relearn the provider capabilities from scratch.

Existing supervised routing systems [7], [8] rely on offline classifiers or cascading rules, lacking the ability to adapt online. Multi-objective bandits [4], [5] maintain vector rewards but assume all users share identical preferences. Multi-task bandits [12], [13] transfer knowledge across related tasks but continue to operate on scalar rewards, preserving the capability–preference entanglement.

TL-LinUCB resolves this by keeping capability and preference strictly separated throughout the learning process. We make three specific contributions:

- 1) **Capability–Preference Decomposition.** Provider capability is estimated from raw outcome observations; preference weights never enter the update equations. Tenant preferences appear only at routing time as a linear projection. Consequently, changing a tenant’s preference vector requires no capability re-estimation.

- 2) **Uncertainty-Guided Global–Local Transfer.** The ratio of global to local estimator uncertainty determines how much each tier influences the routing decision. New tenants draw heavily on the well-conditioned global model, with the blend shifting smoothly toward the local estimate as per-tenant observations accumulate.
- 3) **Controlled Empirical Evaluation.** Four synthetic experiments characterize when the decomposition reduces cumulative pseudo-regret. We also empirically identify the boundary at which bandit exploration overhead eliminates the adaptive benefit entirely.

II. BACKGROUND

A. Supervised and Static LLM Routing

FrugalGPT [7] cascades through increasingly capable models, escalating the query only when generation uncertainty exceeds a learned threshold. RouteLLM [8] classifies queries using pairwise preference data, routing requests to either a frontier or a budget endpoint. Other approaches [9], [10] match queries to providers via benchmark-derived capability scores, while Šakota et al. [11] fit a meta-model over cost–quality trade-offs across model families. These systems all commit to a fixed routing policy at deployment. Consequently, they cannot adjust when provider latencies shift in real time or when tenant service-level agreements change.

B. Contextual Bandits for Online Model Selection

LinUCB [1] estimates expected scalar rewards via regularized linear regression and selects arms using an upper-confidence bonus. Abbasi-Yadkori et al. [2] proved sharp regret bounds for this family, and Thompson-sampling variants followed [3]. Russac et al. [14] extended the framework to non-stationary reward processes using discounted design matrices. More recently, Yue et al. [15] applied contextual bandits directly to LLM provider selection. In all these formulations, the learner maintains a scalar reward model per arm. Applying this directly to a multi-tenant gateway means observations from tenants with incompatible preferences update the identical parameter vector. This *cross-tenant parameter interference* degrades routing accuracy for all tenants simultaneously.

C. Multi-Objective and Transfer Bandits

Multi-objective bandit methods [4]–[6] track vector payoffs to identify Pareto-optimal arms, but they assume a single decision-maker with a fixed preference profile. Multi-task bandits [12], [13] accelerate learning by sharing information across related tasks, typically through a shared prior. However, the per-task reward remains a scalar, meaning the capability–preference coupling persists. Table I maps these distinctions. Among the architectures compared, TL-LinUCB uniquely maintains vector-valued outcome estimates, applies preferences strictly as an inference-time projection, and blends global and local estimators using uncertainty guidance.

III. METHODOLOGY

A. Problem Formulation

An API gateway routes requests from N tenants $\mathcal{I} = \{1, \dots, N\}$ to K LLM providers $\mathcal{A} = \{1, \dots, K\}$. At step t , tenant i submits a prompt; the gateway extracts a context vector $\mathbf{x}_t \in \mathbb{R}^d$ encoding query attributes such as reasoning complexity.

Routing to provider a produces an outcome vector $\mathbf{y}_t \in \mathbb{R}^c$, each component a normalized performance metric (quality, latency efficiency, cost efficiency, availability), with higher values indicating better performance. Expected outcomes are modeled as linear in context:

$$\mathbb{E}[\mathbf{y}_t \mid a, \mathbf{x}_t] = \Theta_a^\top \mathbf{x}_t, \quad (1)$$

where $\Theta_a \in \mathbb{R}^{d \times c}$ is the unknown capability matrix for provider a . Availability is binary but passes through the same estimator as continuous dimensions. Latency and cost are normalized against fixed bounds before experiments begin.

B. Capability–Preference Decomposition

Tenant i specifies a preference vector $\mathbf{w}_i \in \mathbb{R}^c$ constrained by $\sum_j w_{i,j} = 1$ and $w_{i,j} \geq 0$. The core architectural decision is that Θ_a learns exclusively from raw outcome observations rather than scalarized utilities. Tenant preference enters the pipeline exactly once: as the inner product $\mathbf{w}_i^\top \hat{\mathbf{y}}_a$ computed during the routing decision. Because $\mathbf{w}_i$ does not appear in any update equation, revising a tenant’s service-level agreement leaves the capability parameters completely intact.

C. Multivariate Capability Estimation

Two ℓ_2 -regularized linear estimators are maintained per provider a :

- A **global estimator** ($\mathbf{A}_{a,g}, \mathbf{B}_{a,g}$) updated on every observation, regardless of which tenant generated it.
- One **local estimator** ($\mathbf{A}_{a,i}, \mathbf{B}_{a,i}$) per tenant, updated only from that tenant’s observations.

Both produce capability matrices via ridge regression:

$$\Theta_{a,g} = \mathbf{A}_{a,g}^{-1} \mathbf{B}_{a,g}, \quad \Theta_{a,i} = \mathbf{A}_{a,i}^{-1} \mathbf{B}_{a,i}. \quad (2)$$

D. Uncertainty-Guided Global–Local Transfer

The context-direction uncertainty of each estimator is:

$$\sigma_{a,g} = \sqrt{\mathbf{x}_t^\top \mathbf{A}_{a,g}^{-1} \mathbf{x}_t}, \quad \sigma_{a,i} = \sqrt{\mathbf{x}_t^\top \mathbf{A}_{a,i}^{-1} \mathbf{x}_t}. \quad (3)$$

Since $\mathbf{A}_{a,g}$ accumulates all tenants’ observations, $\mathbf{A}_{a,g} \succeq \mathbf{A}_{a,i}$, which implies $\sigma_{a,g} \leq \sigma_{a,i}$. The transfer coefficient is set from this ratio:

$$\alpha(a, i) = \text{clip}\left(1 - \frac{\sigma_{a,g}}{\sigma_{a,i} + \epsilon}, 0, 1\right), \quad (4)$$

where ϵ prevents division by near-zero. When a tenant’s local estimator is new and poorly conditioned, $\sigma_{a,i}$ is large and $\alpha \approx 1$, so routing relies primarily on the global estimate. As local observations accumulate, $\sigma_{a,i}$ falls, α declines toward zero, and the blend shifts to the tenant-specific model. The blended prediction is:

$$\hat{\mathbf{y}}_a = \alpha(a, i) \Theta_{a,g}^\top \mathbf{x}_t + (1 - \alpha(a, i)) \Theta_{a,i}^\top \mathbf{x}_t. \quad (5)$$

TABLE I
COMPARISON OF LLM ROUTING AND BANDIT ARCHITECTURES

Method	Online Adapt.	Vector Payoff	Tenant-Specific	Pref. Shift w/o Relearn	Global-Local Transfer
FrugalGPT [7]	No	No	No	No	No
RouteLLM [8]	No	No	No	No	No
Scalar LinUCB [1]	Yes	No	No	No	No
Online LLM Routing [15]	Yes	No	No	No	No
Multi-Objective Bandits [4]	Yes	Yes	No	Varies	No
Multi-Task Bandits [12]	Yes	No	Yes	No	Yes
TL-LinUCB (Ours)	Yes	Yes	Yes	Yes	Yes

E. Preference Projection and Routing

Tenant i 's projected utility for provider a is $\hat{u}_{a,i} = \mathbf{w}_i^\top \hat{\mathbf{y}}_a$. Because $\hat{\mathbf{y}}_a$ blends two correlated estimators, we use a composite effective uncertainty:

$$\sigma_{\text{eff}} = \sqrt{\alpha(a,i)^2 \sigma_{a,g}^2 + (1 - \alpha(a,i))^2 \sigma_{a,i}^2}. \quad (6)$$

The routing score is $S_{a,i} = \hat{u}_{a,i} + \beta \sigma_{\text{eff}}$, where $\beta = 0.5 \|\mathbf{w}_i\|_2$. Scaling β by the preference norm keeps the exploration bonus proportional to the projected utility scale, ensuring comparable exploration across tenants with different preference magnitudes.

F. Online Update

After routing to a^* and observing $\mathbf{y}_t$, both tiers receive rank-one increments:

$$\mathbf{A}_{a^*,g} \leftarrow \mathbf{A}_{a^*,g} + \mathbf{x}_t \mathbf{x}_t^\top, \quad \mathbf{B}_{a^*,g} \leftarrow \mathbf{B}_{a^*,g} + \mathbf{x}_t \mathbf{y}_t^\top \quad (7)$$

with identical updates to $(\mathbf{A}_{a^*,i}, \mathbf{B}_{a^*,i})$. Matrix inverses are computed explicitly; the Sherman–Morrison formula [16] could replace these with $\mathcal{O}(d^2)$ rank-one updates in latency-sensitive deployments.

G. Non-Stationarity

When tenant i changes its SLA weights $\mathbf{w}_i$, the capability matrices Θ_a remain mathematically valid because they track provider behavior independent of any specific weighting. The new preference vector takes effect immediately through the projection at the very next step. In contrast, provider degradations—such as a sudden latency spike—are absorbed iteratively through ongoing outer-product updates to both global and local matrices as the degraded observations arrive.

H. Theoretical Properties

Conjecture 1 (Regret Bound). *Under the linear reward assumption and bounded observation variance, cumulative expected regret of TL-LinUCB over T steps satisfies $\mathcal{O}(d\sqrt{T} \log(T))$.*

Justification. Three properties suggest plausibility: (i) projecting an unbiased vector estimate onto fixed weights $\mathbf{w}_i$ yields an unbiased scalar estimate; (ii) $\mathbf{A}_{a,g} \succeq \mathbf{A}_{a,i}$ implies the blended estimator's variance is no worse than the local estimator's alone; (iii) observed cumulative regret grows sub-linearly across all experimental conditions.

Algorithm 1 TL-LinUCB Online Routing

Require: Providers K , dimensions d, c , exploration β , stabilizer ϵ

- 1: Initialize $\mathbf{A}_{a,g} = 0.1\mathbf{I}_d$, $\mathbf{B}_{a,g} = \mathbf{0}_{d \times c}$ for all a
- 2: Allocate local matrices $\mathbf{A}_{a,i} = 0.1\mathbf{I}_d$, $\mathbf{B}_{a,i} = \mathbf{0}_{d \times c}$ on tenant arrival
- 3: **for** step $t = 1, 2, \dots$ **do**
- 4: Receive context $\mathbf{x}_t$, tenant i , preference $\mathbf{w}_i$
- 5: **for** each provider a **do**
- 6: Compute $\Theta_{a,g}$, $\Theta_{a,i}$, $\sigma_{a,g}$, $\sigma_{a,i}$, and α
- 7: $\hat{\mathbf{y}}_a \leftarrow \alpha \Theta_{a,g}^\top \mathbf{x}_t + (1 - \alpha) \Theta_{a,i}^\top \mathbf{x}_t$
- 8: $S_{a,i} \leftarrow \mathbf{w}_i^\top \hat{\mathbf{y}}_a + \beta \sqrt{\alpha^2 \sigma_{a,g}^2 + (1 - \alpha)^2 \sigma_{a,i}^2}$
- 9: **end for**
- 10: Select $a^* = \arg \max_a S_{a,i}$; route; observe $\mathbf{y}_t$
- 11: Update $\mathbf{A}_{a^*,g}$, $\mathbf{B}_{a^*,g}$, $\mathbf{A}_{a^*,i}$, $\mathbf{B}_{a^*,i}$ with rank-one increments
- 12: **end for**

A formal proof is not available. Because $\mathbf{A}_{a,g}$ includes the local tenant's observations, the noise terms of the two estimators are correlated, which violates the independence required by standard self-normalized martingale arguments [2]. The term $\beta \sigma_{\text{eff}}$ is therefore an empirically motivated heuristic, not a provably valid confidence bound.

Lemma 1. *If $\alpha(a,i) = 1$ and $\beta = 0$, TL-LinUCB reduces to greedy global vector regression.*

Proof. With $\alpha = 1$ and $\beta = 0$, the routing score reduces to $S_{a,i} = \mathbf{w}_i^\top \Theta_{a,g}^\top \mathbf{x}_t$, which is identical to greedy selection under a single shared capability model. $\square$

Transfer Limitation. When a single high-volume tenant dominates the global design matrix, $\sigma_{a,g}$ shrinks toward zero, driving $\alpha \rightarrow 1$ and effectively imposing the global estimate on all tenants regardless of local evidence. Minority tenants with different capability environments converge more slowly to their correct local routing policies. The mechanism works best when capability profiles are broadly similar across tenants.

I. Algorithm and Complexity

Algorithm 1 formalizes the procedure. Per step, evaluating K providers costs $\mathcal{O}(Kd^3 + Kcd^2)$ with explicit matrix inversion; Sherman–Morrison updates reduce this to $\mathcal{O}(Kd^2)$ in practice. Memory scales as $\mathcal{O}(NK(d^2 + cd))$.

IV. EVALUATION

A. Synthetic Environment

We evaluate with $K = 5$ synthetic provider profiles mimicking the trade-off structure of GPT-4o, Claude 3.5 Sonnet, Llama-3-70B, Gemini 1.5 Flash, and Mistral-Nemo-12B. Each routing step returns a four-component outcome vector ($c = 4$): quality, latency efficiency, cost efficiency, and availability, drawn from linear-Gaussian models over a $d = 16$ -dimensional context. Frontier-class profiles reach quality scores up to 0.98 on complex queries but impose latency above 680 ms and high token costs; lightweight profiles respond near 150ms at a fraction of the cost, with quality degrading on complex inputs. Latency efficiency is normalized within [100ms, 1200ms]; cost efficiency against a \$0.012/1k-token ceiling; availability is a Bernoulli draw.

B. Baselines

- **Scalar Monolithic LinUCB:** Single pooled estimator trained on scalarized utility $\mathbf{w}_i^\top \mathbf{y}_t$ across all tenants. Maximum sharing; maximum cross-tenant interference.
- **Scalar Isolated LinUCB:** Per-tenant scalar estimators with no cross-tenant sharing. Eliminates interference; each tenant handles its own cold-start.
- **Vector Global-Only / Local-Only:** Single-tier ablations retaining the vector representation but omitting the global-local blend, isolating each mechanism’s contribution.
- **Fixed- α Transfer:** TL-LinUCB with α fixed at 0.5, testing whether dynamic uncertainty weighting adds value over a constant blend.
- **Static Selection:** Deterministic routing to a single provider throughout.

All bandit baselines share the same ℓ_2 -regularized ridge regression, identical context sequences, and fixed noise realizations across paired seeds. The exploration parameter is $\beta = 0.5\|\mathbf{w}_i\|_2$ for all LinUCB variants.

C. Experimental Design

Experiment 1 (Baseline Ablation). Five tenants with heterogeneous, conflicting preference vectors over $T = 3,000$ steps, 30 paired seeds. Measures the separate contributions of the capability-preference decomposition and the transfer mechanism.

Experiment 2 (Preference Shift). A single tenant shifts $\mathbf{w}_i$ at steps 2,000 and 4,000 over $T = 6,000$ (10 seeds), cycling quality-dominant $\rightarrow$ latency-dominant $\rightarrow$ cost-dominant. Provider capabilities are held fixed, isolating adaptation cost.

Experiment 3 (Provider Drift). The GPT-4o-inspired provider incurs a 1,500 ms latency increase at $t = 2,000$ ($T = 4,000$, 10 seeds). Tests whether accumulated matrix statistics redirect traffic after a sudden provider degradation.

Experiment 4 (Model Mismatch). Linear-Gaussian outcomes replaced with sigmoidal quality saturation and heavy-tailed latency ($T = 3,000$, 10 seeds). Tests robustness when the linear assumption is violated.

TABLE II
EXPERIMENTAL CONFIGURATION

Parameter	Value
Providers (K)	5 (Pareto-distributed synthetic profiles)
Context / Outcome dims. (d, c)	16, 4 (Quality, Latency, Cost, Avail.)
Exp 1: T , Seeds	3000, 30 paired
Exp 2: T , Seeds	6000, 10 paired
Exp 3: T , Seeds	4000, 10 paired
Exp 4: T , Seeds	3000, 10 paired
Exploration β	$0.5\ \mathbf{w}\ _2$
Normalization	Fixed from system configuration

D. Metrics

The primary metric is cumulative pseudo-regret. At each step t , an oracle selects $a_t^* = \arg \max_a \mathbf{w}_i^\top \mathbb{E}[\mathbf{y}_t \mid a, \mathbf{x}_t]$ from the true distribution before noise is applied, and regret accumulates the gap to the deployed policy:

$$\mathcal{R}(T) = \sum_{t=1}^T (\mathbf{w}_i^\top \mathbb{E}[\mathbf{y}_t \mid a_t^*, \mathbf{x}_t] - \mathbf{w}_i^\top \mathbb{E}[\mathbf{y}_t \mid a_t, \mathbf{x}_t]). \quad (8)$$

Random seeds, context sequences, provider parameters, and noise realizations are fixed and shared identically across all compared policies within each seed pair.

V. RESULTS AND DISCUSSION

A. Capability-Preference Decomposition (Exp. 1)

Table III details the pseudo-regret after 3,000 steps across 30 random seeds. The Scalar Monolithic baseline trains on scalarized utilities pooled from five tenants with conflicting preferences, accumulating 246.35 ± 5.77 in regret. This high regret reflects the consequence of cross-tenant interference: when incompatible objectives update a single scalar parameter, the converged model serves no individual tenant well.

Switching to a vector-valued global estimator (Vector Global-Only) reduces mean regret to 213.35 ± 5.52 , a 13.4% improvement. Here, each outcome dimension is estimated independently, and preferences enter only at decision time. Consequently, no preference signal corrupts the capability update. A Wilcoxon signed-rank test confirms the improvement is highly systematic across seeds ($p = 1.86 \times 10^{-9}$, Cohen’s $d = 5.84$). Enabling the dynamic α coefficient (TL-LinUCB) further reduces regret to 209.94 ± 5.94 .

B. Preference Shift Adaptation (Exp. 2)

Experiment 2 runs a single tenant through three preference regimes over 6,000 steps, with transitions at steps 2,000 and 4,000. The Scalar Isolated baseline reached $1,032.99 \pm 26.13$: each preference shift invalidates the scalar reward model, and the estimator must relearn a routing policy from scratch before it can route competitively again. Resetting parameters at each shift reduces this to 537.47 ± 13.69 , eliminating the relearning lag at the cost of discarding accumulated capability knowledge.

TL-LinUCB accumulated 430.86 ± 24.72 , a 19.8% reduction compared to the reset baseline. Because provider capabilities

TABLE III
CUMULATIVE PSEUDO-REGRET (30 SEEDS, $T = 3000$) — EXP. 1

Routing Policy	Mean	Std.
<i>Ablation A: Capability–Preference Decomposition</i>		
Scalar Monolithic (Pooled)	246.35	± 5.77
Scalar Isolated (Siloed)	220.90	± 4.77
Vector Local-Only	220.90	± 4.77
Vector Global-Only	213.35	± 5.52
<i>Ablation B: Global–Local Transfer</i>		
TL-LinUCB (Fixed $\alpha = 0.5$)	218.31	± 4.53
TL-LinUCB (Dynamic α)	209.94	± 5.94
<i>Static Baseline</i>		
Static (Best Arm: Llama-3-70B)	195.36	± 1.26

remained unchanged throughout this experiment, shifting $\mathbf{w}_i$ altered only the projection $\hat{\mathbf{u}} = \mathbf{w}_{\text{new}}^\top \hat{\mathbf{y}}$. This mathematical separation allows the system to adapt to the new SLA at the very next step without any parameter re-estimation.

C. Transfer Mechanism (Exp. 1)

Vector Local-Only and Scalar Isolated both reach 220.90: without cross-tenant sharing, ridge regression under identical regularization produces mathematically equivalent predictions in the two configurations. TL-LinUCB with dynamic α (209.94) outperforms both and the Fixed- $\alpha = 0.5$ variant (218.31). The margin over the fixed coefficient is modest because five tenants sharing a global estimator over 3,000 steps produce a well-conditioned global model—leaving little room for dynamic weighting to matter. The dynamic mechanism is most valuable when tenants enter at different times and observation counts are highly unequal.

D. Provider Drift and Model Mismatch (Exps. 3–4)

Provider Drift. Following a 1,500 ms latency spike at step 2,000, the GPT-4o-inspired provider’s traffic share fell from 83.3% (steps 1,500–2,000) to 0.0% (steps 2,500–3,000). The degraded latency observations lowered its estimated efficiency in both matrix tiers, dropping its routing score below competing providers. Total regret: 288.06 ± 11.78 over 4,000 steps. The transition is gradual rather than instantaneous: outer-product updates are incremental, so several degraded observations must accumulate before $\Theta_{a,i}$ and $\Theta_{a,g}$ fully reflect the new provider state. This creates a transient regret period between the spike and the full traffic redirect, which is characteristic of any observation-driven estimator without explicit change-point detection.

Model Mismatch. Under sigmoidal quality saturation and heavy-tailed latency noise, TL-LinUCB reached 135.52 ± 9.93 versus 258.26 ± 5.87 for Scalar Isolated. The relative advantage persists but narrows: as the linear approximation fits the true process less accurately, the precision benefit of separate dimension tracking diminishes. The fact that TL-LinUCB still outperforms Scalar Isolated in this condition suggests the decomposition provides some residual benefit even under misspecification, but this should not be interpreted as evidence of robustness to arbitrary nonlinearity.

E. Ablation Interpretation and Design Trade-offs

The ablation structure in Table III separates two independent design axes: (A) whether capability estimation is scalar or vector-valued, and (B) whether knowledge is shared globally, isolated per tenant, or blended. Reading Ablation A vertically, moving from Scalar Monolithic (246.35) to Scalar Isolated (220.90) to Vector Global-Only (213.35) reveals that eliminating cross-tenant gradient interference and then decomposing into vector-valued estimates each contribute distinct, additive regret reductions. Reading Ablation B, the dynamic transfer coefficient (209.94) outperforms both the fixed- α variant (218.31) and the isolated baseline (220.90), confirming that the variance-ratio mechanism adds value beyond a naive constant blend.

The equal regret of Vector Local-Only and Scalar Isolated (220.90) is not a coincidence. With identical ℓ_2 regularization and no cross-tenant sharing, the inner product of the ridge-regression coefficient with the preference vector is algebraically equivalent to first scalarizing and then regressing. The vector representation only becomes advantageous when global information is shared—as in Vector Global-Only—or when the preference vector changes without a corresponding provider change—as in Experiment 2. This means the decomposition’s value is conditional on either cross-tenant sharing being active or preferences being non-stationary.

F. Boundary Condition and Practical Guidance

Static routing to the Llama-3-70B profile achieves 195.36 ± 1.26 , below every adaptive policy in Experiment 1, including TL-LinUCB. This occurs because the profile is near-optimal for the lightweight-query workload that dominates the synthetic setup, and any bandit strategy incurs exploration overhead from sampling sub-optimal providers during parameter estimation. The boundary condition is not specific to TL-LinUCB—it applies to any contextual-bandit routing policy—but it is important to state explicitly: active exploration only pays off when the provider landscape offers genuine, context-dependent trade-offs that a static policy cannot exploit.

In practice, this suggests a pragmatic deployment approach: run a short static-routing warmup to establish a performance floor before enabling adaptive routing. Alternatively, annealing the exploration parameter β over time reduces exploration as capability estimates stabilize. The regret figures in Experiment 1 should therefore be interpreted as upper bounds on the cost of exploration for the given workload and provider mix, not as evidence that adaptive routing is always preferable to static assignment.

G. Limitations

Several limitations constrain these findings. The synthetic environment exhibits Pareto trade-offs but omits production complexities like rate limiting, context-dependent pricing, and non-independent infrastructure availability patterns. Whether the linear-Gaussian capability model is a reasonable approximation requires evaluation on real API traces.

Conjecture 1 remains unproved. Because the global design matrix $\mathbf{A}_{a,g}$ includes the local tenant’s own observations, the noise terms of $\Theta_{a,g}$ and $\Theta_{a,i}$ are correlated. This violates the independence condition required by the self-normalized martingale concentration inequalities underlying LinUCB regret bounds [2]. The exploration term $\beta\sigma_{\text{eff}}$ is therefore a practical heuristic. The sub-linear regret growth observed empirically is consistent with the conjectured rate but does not constitute a theoretical guarantee.

The variance-ratio transfer mechanism can become counter-productive when one high-volume tenant dominates the global design matrix, driving $\alpha \rightarrow 1$ and overriding local evidence for minority tenants. Characterizing this failure mode and designing a regularization strategy to limit dominance effects remains an open problem. Finally, explicit $d \times d$ matrix inversion at each step is acceptable for $d = 16$ but becomes a bottleneck for higher-dimensional contexts, requiring Sherman–Morrison updates in production deployments.

VI. CONCLUSION

TL-LinUCB is built on a simple structural observation: in a multi-tenant gateway, provider capability and tenant preference are different kinds of information that should not be mixed during parameter estimation. Capability—what a provider reliably delivers for a given query—is a property of the provider and the query context. Preference—how much a tenant weights quality against latency or cost—is a property of the tenant and can change without any provider changing its behavior. When these are combined in a single scalar update, every preference change invalidates accumulated capability knowledge. Keeping them separate means it does not.

The implementation is architecturally minimal. Capability is tracked as a ridge-regression coefficient matrix updated from raw outcome vectors; preference enters as a dot product at routing time only. Two estimator tiers—one global, one per tenant—are blended by a transfer coefficient derived from their relative uncertainty in the direction of the current context, providing a principled cold-start mechanism without requiring explicit modeling of tenant similarity.

The empirical evaluation supports the separation’s value. Across 30 seeds with five synthetic profiles, the decomposition reduces cumulative pseudo-regret by 13.4% over a scalar monolithic baseline. Under abrupt preference drift, the regret reduction over a reset scalar policy reaches 19.8%. Both figures reflect a consistent mechanism: projection updates take effect immediately, whereas scalar re-estimation requires new observations to converge. The evaluation also establishes that when one provider is near-optimal for the workload, any bandit’s exploration overhead eliminates its adaptive benefit, making static routing preferable. This boundary is a general property of online exploration strategies.

Three directions remain open. Replacing the linear capability model with a nonlinear function approximator (e.g., kernel methods) could extend the architecture to settings where the linear assumption fails. Evaluation on real LLM API trace data is necessary to confirm whether synthetic Pareto trade-offs

reflect production conditions. Finally, a formal regret analysis resolving Conjecture 1 requires new concentration inequalities accommodating correlated estimators, or a reformulated transfer mechanism restoring the independence condition. Until then, $\beta\sigma_{\text{eff}}$ should be validated empirically rather than treated as a certified confidence bound.

REFERENCES

- [1] L. Li, W. Chu, J. Langford, and R. E. Schapire, “A contextual-bandit approach to personalized news article recommendation,” in *Proc. 19th Int. Conf. World Wide Web (WWW)*, 2010, pp. 661–670.
- [2] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, “Improved algorithms for linear stochastic bandits,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2011, pp. 2312–2320.
- [3] S. Agrawal and N. Goyal, “Thompson sampling for contextual bandits with linear payoffs,” in *Proc. 30th Int. Conf. Mach. Learn. (ICML)*, 2013, pp. 127–135.
- [4] M. M. Dragan and A. Nowe, “Designing multi-objective multi-armed bandits algorithms: A study,” in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2013, pp. 1–8.
- [5] D. M. Roijers, P. Vamplew, S. Whiteson, and R. Dazeley, “A survey of multi-objective sequential decision-making,” *J. Artif. Intell. Res. (JAIR)*, vol. 48, pp. 67–113, 2013.
- [6] R. Busa-Fekete, B. Szörényi, P. Weng, W. Cheng, and E. Hüllermeier, “Preference-based reinforcement learning: evolutionary and statistical approaches,” *Machine Learning*, vol. 97, no. 3, pp. 327–351, 2014.
- [7] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] I. Ong *et al.*, “RouteLLM: Learning to route LLMs with preference data,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [9] T. Shnitzer *et al.*, “Large language model routing with benchmark datasets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] A. Mohammadshahi *et al.*, “Routoo: Learning to route to large language models effectively,” *Preprint*, 2024.
- [11] M. Šakota, M. G. Rabbat, and B. Kveton, “Fly-swat or cannon? Cost-effective language model choice via meta-modeling,” *Preprint*, 2024.
- [12] B. Kveton *et al.*, “Meta-Thompson sampling,” in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 5884–5893.
- [13] A. Deshmukh, U. Srinivasan, and S. Bhowmick, “Multi-task learning for contextual bandits,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [14] P. Rousset, C. Vernade, and O. Cappé, “Weighted linear bandits for non-stationary environments,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [15] K. Yue *et al.*, “Large language model routing with benchmark updates,” *Preprint*, 2024.
- [16] J. Sherman and W. J. Morrison, “Adjustment of an inverse matrix corresponding to a change in one element of a given matrix,” *Annals of Mathematical Statistics*, vol. 21, no. 1, pp. 124–127, 1950.